AIAC_LAB_ASSIGNMENT_16.3

Assignment 16.3

Field	Details
Name	Maripelli Chodhitha
HT No	2303A52039
Batch	31

Task 1 — Hospital Management Database

Requirement

Create a normalized SQLite schema and SQL queries (using JOINs) for a **Hospital Management System** with the following tables:

- Doctors
- Patients
- Appointments

Include constraints such as:

- Unique IDs (primary keys)
- Valid relationships (foreign keys)
- Valid dates (NOT NULL, basic checks where applicable)

SQLite Schema + Sample Data + Queries

```sql
-- Drop tables if they exist (order matters due to FK depende
ncies)
DROP TABLE IF EXISTS Appointments;
DROP TABLE IF EXISTS Doctors;
DROP TABLE IF EXISTS Patients;
```

```sql
-- Doctors table
CREATE TABLE Doctors (
    doctor_id INTEGER PRIMARY KEY,
    name      TEXT NOT NULL,
    specialty TEXT NOT NULL
);

-- Patients table
CREATE TABLE Patients (
    patient_id    INTEGER PRIMARY KEY,
    name          TEXT NOT NULL,
    date_of_birth TEXT NOT NULL
        -- Basic format check (SQLite stores dates as TEXT)
        CHECK (date_of_birth GLOB '[0-9][0-9][0-9][0-9]-[0-1]
[0-9]-[0-3][0-9]')
);

-- Appointments table
CREATE TABLE Appointments (
    appointment_id   INTEGER PRIMARY KEY,
    doctor_id        INTEGER NOT NULL,
    patient_id       INTEGER NOT NULL,
    appointment_date TEXT NOT NULL
        CHECK (appointment_date GLOB '[0-9][0-9][0-9][0-9]-[0
-1][0-9]-[0-3][0-9]'),
    FOREIGN KEY (doctor_id)  REFERENCES Doctors(doctor_id),
    FOREIGN KEY (patient_id) REFERENCES Patients(patient_id)
);

-- Sample data
INSERT INTO Doctors (name, specialty) VALUES
    ('Dr. Smith',   'Cardiology'),
    ('Dr. Johnson', 'Neurology'),
    ('Dr. Lee',     'Pediatrics');

INSERT INTO Patients (name, date_of_birth) VALUES
```

```sql
    ('Alice Brown',   '1980-05-15'),
    ('Bob Green',     '1990-08-22'),
    ('Charlie White', '2000-12-01');

INSERT INTO Appointments (doctor_id, patient_id, appointment_
date) VALUES
    (1, 1, '2023-10-15'),
    (1, 2, '2023-10-16'),
    (2, 1, '2023-10-17');

-- Query 1: List all appointments for a specific doctor (by d
octor_id)
SELECT
    d.doctor_id,
    d.name AS doctor_name,
    a.appointment_date,
    p.patient_id,
    p.name AS patient_name
FROM Appointments a
JOIN Doctors d  ON a.doctor_id  = d.doctor_id
JOIN Patients p ON a.patient_id = p.patient_id
WHERE d.doctor_id = 1
ORDER BY a.appointment_date;

-- Query 2: Retrieve patient history by patient_id
SELECT
    p.patient_id,
    p.name AS patient_name,
    p.date_of_birth,
    d.doctor_id,
    d.name AS doctor_name,
    a.appointment_date
FROM Patients p
JOIN Appointments a ON p.patient_id = a.patient_id
JOIN Doctors d      ON a.doctor_id  = d.doctor_id
WHERE p.patient_id = 1
```

```sql
ORDER BY a.appointment_date;

-- Query 3: Count total patients treated by each doctor
SELECT
    d.doctor_id,
    d.name AS doctor_name,
    COUNT(DISTINCT a.patient_id) AS total_patients_treated
FROM Doctors d
LEFT JOIN Appointments a ON d.doctor_id = a.doctor_id
GROUP BY d.doctor_id, d.name
ORDER BY total_patients_treated DESC;
```

Task 2 — Real-Time Application: Online Shopping Database

Requirement

Design a normalized SQLite schema for an **E-commerce platform** with:

- Users
- Products
- Orders
- OrderDetails

Write queries (using JOINs/aggregations):

- Retrieve all orders by a user
- Find the most purchased product
- Calculate total revenue in a given month

Also include:

- Normalization notes (brief)
- Indexing suggestions for optimization

SQLite Schema + Sample Data + Queries + Indexing

```sql
-- Drop tables if they exist
DROP TABLE IF EXISTS OrderDetails;
DROP TABLE IF EXISTS Orders;
DROP TABLE IF EXISTS Products;
DROP TABLE IF EXISTS Users;

-- Users table
CREATE TABLE Users (
    user_id   INTEGER PRIMARY KEY,
    username  TEXT NOT NULL,
    email     TEXT NOT NULL UNIQUE
);

-- Products table
CREATE TABLE Products (
    product_id   INTEGER PRIMARY KEY,
    product_name TEXT NOT NULL,
    price        REAL NOT NULL CHECK (price >= 0)
);

-- Orders table
CREATE TABLE Orders (
    order_id   INTEGER PRIMARY KEY,
    user_id    INTEGER NOT NULL,
    order_date TEXT NOT NULL,
    FOREIGN KEY (user_id) REFERENCES Users(user_id)
);

-- OrderDetails table (line items)
CREATE TABLE OrderDetails (
    order_detail_id INTEGER PRIMARY KEY,
    order_id        INTEGER NOT NULL,
    product_id      INTEGER NOT NULL,
    quantity        INTEGER NOT NULL CHECK (quantity > 0),
    FOREIGN KEY (order_id)   REFERENCES Orders(order_id),
```

```sql
    FOREIGN KEY (product_id) REFERENCES Products(product_id)
);

-- Sample data
INSERT INTO Users (username, email) VALUES
    ('Alice', 'alice@example.com'),
    ('Bob',   'bob@example.com');

INSERT INTO Products (product_name, price) VALUES
    ('Laptop', 1000.00),
    ('Mouse',    25.00);

INSERT INTO Orders (user_id, order_date) VALUES
    (1, '2023-01-15'),
    (2, '2023-01-16');

INSERT INTO OrderDetails (order_id, product_id, quantity) VAL
UES
    (1, 1, 1),
    (1, 2, 2),
    (2, 2, 1);

-- Query 1: Orders by user (by user_id)
SELECT
    u.user_id,
    u.username,
    o.order_id,
    o.order_date
FROM Users u
JOIN Orders o ON u.user_id = o.user_id
WHERE u.user_id = 1
ORDER BY o.order_date;

-- Query 2: Most purchased product (by total quantity)
SELECT
    p.product_id,
```

```sql
    p.product_name,
    SUM(od.quantity) AS total_quantity
FROM Products p
JOIN OrderDetails od ON p.product_id = od.product_id
GROUP BY p.product_id, p.product_name
ORDER BY total_quantity DESC
LIMIT 1;

-- Query 3: Monthly revenue (YYYY-MM)
SELECT
    strftime('%Y-%m', o.order_date) AS month,
    SUM(p.price * od.quantity)      AS revenue
FROM Orders o
JOIN OrderDetails od ON o.order_id = od.order_id
JOIN Products p      ON od.product_id = p.product_id
WHERE strftime('%Y-%m', o.order_date) = '2023-01'
GROUP BY month;

-- Indexing suggestions
CREATE INDEX idx_orders_user_id             ON Orders(user_i
d);
CREATE INDEX idx_order_details_order_id     ON OrderDetails(o
rder_id);
CREATE INDEX idx_order_details_product_id   ON OrderDetails(p
roduct_id);
```

Notes (Normalization + Optimization)

- `OrderDetails` is used to represent the **one-to-many** relationship between `Orders` and purchased products, preventing repeated product columns inside `Orders`.
- Suggested indexes speed up joins and filters on foreign keys (common in analytics queries).

Task 3 — Library Database

Requirement

Design a SQLite schema for a **Library Management System** with:

- Books
- Members
- Loans

Write queries:

- Retrieve all books currently issued
- Find overdue books (loan date > 30 days)
- Count the number of books loaned by each member

Include indexing suggestions.

SQLite Schema + Sample Data + Queries + Indexing

```sql
-- Drop tables if they exist
DROP TABLE IF EXISTS Loans;
DROP TABLE IF EXISTS Books;
DROP TABLE IF EXISTS Members;

-- Books table
CREATE TABLE Books (
    book_id        INTEGER PRIMARY KEY,
    title          TEXT NOT NULL,
    author         TEXT NOT NULL,
    published_year INTEGER
);

-- Members table
CREATE TABLE Members (
    member_id INTEGER PRIMARY KEY,
    name      TEXT NOT NULL,
    join_date TEXT
);
```

```sql
-- Loans table
CREATE TABLE Loans (
    loan_id     INTEGER PRIMARY KEY,
    book_id     INTEGER NOT NULL,
    member_id   INTEGER NOT NULL,
    loan_date   TEXT NOT NULL,
    return_date TEXT,
    FOREIGN KEY (book_id)   REFERENCES Books(book_id),
    FOREIGN KEY (member_id) REFERENCES Members(member_id)
);

-- Sample data
INSERT INTO Books (title, author, published_year) VALUES
    ('The Great Gatsby',       'F. Scott Fitzgerald', 1925),
    ('To Kill a Mockingbird',  'Harper Lee',          1960),
    ('1984',                   'George Orwell',       1949);

INSERT INTO Members (name, join_date) VALUES
    ('Alice Johnson',  '2020-01-15'),
    ('Bob Smith',      '2021-06-10'),
    ('Charlie Brown',  '2019-11-05');

INSERT INTO Loans (book_id, member_id, loan_date, return_dat
e) VALUES
    (1, 1, '2023-01-10', NULL),
    (2, 2, '2023-02-20', '2023-03-01'),
    (3, 3, '2023-01-05', NULL);

-- Query 1: Currently issued books (return_date is NULL)
SELECT
    b.title,
    m.name  AS member_name,
    l.loan_date
FROM Loans l
JOIN Books b   ON l.book_id   = b.book_id
```

```sql
JOIN Members m ON l.member_id = m.member_id
WHERE l.return_date IS NULL;

-- Query 2: Overdue books (> 30 days, not returned)
SELECT
    b.title,
    m.name  AS member_name,
    l.loan_date
FROM Loans l
JOIN Books b   ON l.book_id   = b.book_id
JOIN Members m ON l.member_id = m.member_id
WHERE l.return_date IS NULL
  AND julianday('now') - julianday(l.loan_date) > 30;

-- Query 3: Count of books loaned by each member
SELECT
    m.member_id,
    m.name AS member_name,
    COUNT(l.loan_id) AS books_loaned
FROM Members m
LEFT JOIN Loans l ON m.member_id = l.member_id
GROUP BY m.member_id, m.name
ORDER BY books_loaned DESC;

-- Indexing suggestions
CREATE INDEX idx_loans_book_id    ON Loans(book_id);
CREATE INDEX idx_loans_member_id  ON Loans(member_id);
CREATE INDEX idx_loans_loan_date  ON Loans(loan_date);
CREATE INDEX idx_loans_return_date ON Loans(return_date);
```

Task 4 — Query Optimization (JOIN vs Subquery)

Given (Subquery)

```sql
SELECT *
FROM Books
WHERE author_id IN (SELECT author_id FROM Authors WHERE count
ry = 'UK');
```

Optimized (JOIN)

```sql
SELECT b.*
FROM Books b
JOIN Authors a ON b.AuthorID = a.AuthorID
WHERE a.Country = 'UK';
```

SQLite Demo (Schema + Data)

```sql
DROP TABLE IF EXISTS Books;
DROP TABLE IF EXISTS Authors;

CREATE TABLE Authors (
    AuthorID INTEGER PRIMARY KEY,
    Name     TEXT NOT NULL,
    Country  TEXT NOT NULL
);

CREATE TABLE Books (
    BookID         INTEGER PRIMARY KEY,
    Title          TEXT NOT NULL,
    AuthorID       INTEGER NOT NULL,
    PublishedYear  INTEGER,
    FOREIGN KEY (AuthorID) REFERENCES Authors(AuthorID)
);

INSERT INTO Authors (Name, Country) VALUES
    ('J.K. Rowling',      'UK'),
    ('George Orwell',     'UK'),
```

```sql
    ('F. Scott Fitzgerald','USA');

INSERT INTO Books (Title, AuthorID, PublishedYear) VALUES
    ('Harry Potter and the Sorcerer''s Stone', 1, 1997),
    ('1984',                                   2, 1949),
    ('The Great Gatsby',                       3, 1925);

-- Optimized query result
SELECT b.*
FROM Books b
JOIN Authors a ON b.AuthorID = a.AuthorID
WHERE a.Country = 'UK';
```

Task 5 — University Course Registration

Requirement

Design a course registration system for SR University with:

- Students
- Courses
- Faculty
- Registrations

Relationships:

- Each student can register for multiple courses
- Each course is taught by a faculty member

Write queries:

- List all students enrolled in a specific course
- Find faculty members teaching more than 2 courses
- Retrieve courses with the highest number of registrations

SQLite Schema + Sample Data + Queries

```sql
-- Drop tables if they exist
DROP TABLE IF EXISTS Registrations;
DROP TABLE IF EXISTS Students;
DROP TABLE IF EXISTS Courses;
DROP TABLE IF EXISTS Faculty;

CREATE TABLE Faculty (
    faculty_id INTEGER PRIMARY KEY,
    name       TEXT NOT NULL
);

CREATE TABLE Courses (
    course_id   INTEGER PRIMARY KEY,
    course_name TEXT NOT NULL,
    faculty_id  INTEGER NOT NULL,
    FOREIGN KEY (faculty_id) REFERENCES Faculty(faculty_id)
);

CREATE TABLE Students (
    student_id INTEGER PRIMARY KEY,
    name       TEXT NOT NULL
);

CREATE TABLE Registrations (
    registration_id INTEGER PRIMARY KEY,
    student_id      INTEGER NOT NULL,
    course_id       INTEGER NOT NULL,
    FOREIGN KEY (student_id) REFERENCES Students(student_id),
    FOREIGN KEY (course_id)  REFERENCES Courses(course_id)
);

-- Sample data
INSERT INTO Faculty (name) VALUES
    ('Dr. Smith'), ('Dr. Johnson'), ('Dr. Lee');
```

```sql
INSERT INTO Courses (course_name, faculty_id) VALUES
    ('Math 101',    1),
    ('Math 103',    1),
    ('History 201', 2),
    ('Science 301', 3),
    ('Math 102',    1),
    ('History 202', 2),
    ('Science 302', 3);

INSERT INTO Students (name) VALUES
    ('Alice'), ('Bob'), ('Charlie');

INSERT INTO Registrations (student_id, course_id) VALUES
    (1, 1), (1, 2), (2, 1), (3, 3), (2, 3), (3, 1), (3, 2);

-- Query 1: Students in a specific course (by course name)
SELECT
    s.student_id,
    s.name AS student_name,
    c.course_name
FROM Students s
JOIN Registrations r ON s.student_id = r.student_id
JOIN Courses c       ON r.course_id  = c.course_id
WHERE c.course_name = 'Math 101'
ORDER BY s.name;

-- Query 2: Faculty teaching more than 2 courses
SELECT
    f.faculty_id,
    f.name AS faculty_name,
    COUNT(c.course_id) AS course_count
FROM Faculty f
JOIN Courses c ON f.faculty_id = c.faculty_id
GROUP BY f.faculty_id, f.name
HAVING COUNT(c.course_id) > 2
ORDER BY course_count DESC;
```

```sql
-- Query 3: Most registered courses (top 1)
SELECT
    c.course_id,
    c.course_name,
    COUNT(r.registration_id) AS registration_count
FROM Courses c
JOIN Registrations r ON c.course_id = r.course_id
GROUP BY c.course_id, c.course_name
ORDER BY registration_count DESC
LIMIT 1;
```